COMP 018 • DATABASE ADMINISTRATION

PUP-SPOT

Lost and Found System

System Development and Repository Deployment
Progress Documentation

Document information	Details
Course	COMP 018 – Database Administration
Activity	Activity 3 – System Development and Repository Deployment
Database platform	PostgreSQL
Group	Group 5
Repository	https://github.com/grp05-dba/grp05_lostandfound
Documentation branch	docs/project-documentation (reference only; not edited)
Prepared	November 3, 2026
Last updated	November 5, 2026 (PHT)
Reviewed main	9a803d2771610166e638eeba24321764377de915

Group Members and Assigned Responsibilities

Member name	Assigned role / contribution
Daniel Cueto	Developer
Justine Philip Raguin	Co-Developer
Aaron Paul Imperial	Co-Developer
Jenver Napenas	Documentation
Emmanuel Manansala	Co-Developer

Class tracker URL: https://docs.google.com/spreadsheets/d/15msFD5oWM6FzkEOyz3ayJP17TUayv2Qp/edit

Document Scope and Source Basis

This document was updated on November 5, 2026 (Philippine time) against main revision 9a803d2771610166e638eeba24321764377de915. The latest reviewed commit is dated November 5, 2026 at 11:43 AM PHT. It builds on the November 4 update (revision 78ab3d9, 11:23 PM PHT), which in turn updated the October 31 implementation baseline (17dfae5) described in the report prepared on November 3. Section 13 records the timeframe and content changes.

PUP-SPOT now includes PHP API workflows for creating, viewing and updating records, code-based report cancellation, potential matching, search, images, and notification integration. Since November 5, main also contains browser-based interfaces: a public portal (public/app.html) and a marshal console (admin/app.html). These features are present in the reviewed main source. User authentication, successful fresh installation of this exact revision, browser testing of the new interfaces, and live message delivery are not established as verified by this update.

Repository: https://github.com/grp05-dba/grp05_lostandfound

Reviewed main source: https://github.com/grp05-dba/grp05_lostandfound/tree/9a803d2771610166e638eeba24321764377de915

Sections in this report

- System overview and current scope
- Technology stack and architecture
- Database design and SQL initialization
- Implemented API routes and current behavior
- Application roles and intended database permissions
- Development milestones and verification evidence
- Installation and configuration instructions
- Known issues, limitations, and remaining work
- Cross-group verification, AI usage declaration, and dated document update history

1. System Overview

Project name: PUP-SPOT – Lost and Found System

Purpose. PUP-SPOT supports campus lost-and-found reporting, found-item registration, claim handling and possible matches. The database now includes the five core record tables plus three image metadata tables. Files are stored separately under storage/.

Current implemented scope. Public endpoints list and search unclaimed items, read categories, submit lost reports and claims, retrieve submissions by reference code, cancel active lost reports, and display matches and permitted images. Admin endpoints retrieve records, create found items, PATCH items/reports/claims, and read category and SMS gateway information. Matching and notification calls are integrated in services. Since November 5, admin endpoints also upload and delete found-item images and serve item and lost-report images, the public API can remove a potential match using the report code, claim lookup returns the claimed item, and both interfaces are available as browser pages.

Intended broader workflow. Student/guest, campus staff/marshal and DBA responsibilities remain the intended roles. Separate public and admin API processes and database credential settings now exist, but the source still has no application login middleware. Account provisioning, grants and deployment restrictions require verification.

Current Implementation Status

Area	Status based on reviewed main

Area	Status based on reviewed main
PHP API and database connections	Implemented: public/index.php and admin/index.php use separate PDO credential settings.
Read operations and search	Implemented: records, categories, unclaimed items, allowed search filters and match lookup.
SMS notifications	Implemented service calls through TextBee; live delivery not verified in this update.
Email notifications	EmailJS is wired through EmailService; live delivery not verified in this update.
Create and update	POST creation and code lookup/cancellation; admin PATCH for items, reports and claims. No general DELETE endpoint.
Potential matching	Implemented weighted similarity, stored scores, status/date eligibility and refresh/cleanup logic.
Images	Implemented metadata tables, file storage, upload checks and image retrieval.
Frontend and login	Frontend implemented November 5: public portal (public/app.html) and marshal console (admin/app.html). Still no application login middleware in reviewed main.
Role enforcement	Separate connection settings exist and a sample role-creation script was added November 5; applying and testing the grants still needs verification. public/index.php currently opens the admin connection.

2. Technology Stack and Architecture

Component	Repository evidence and role
Programming language	Native PHP with explicit requires.
Database	PostgreSQL; grp05_lostandfounddb; eight declared tables.
PHP extensions	pdo_pgsql, pgsql, curl, mbstring and fileinfo; getimagesize for image verification.
HTTP interface	JSON API, plus image binary responses and static HTML/CSS/JS pages; separate public/admin development servers.
Frontend	Plain HTML, CSS and JavaScript: public/app.html, css/app.css, js/app.js and js/potentialMatching.js (public portal); admin/app.html, css/admin.css and js/admin.js (marshal console). Added November 5.
External integrations	TextBee and EmailJS service calls for configured notification preferences.
Testing tools	Postman, Thunder Client or equivalent; historical Newman/PHP checks are dated below.
Version control	Git/GitHub feature branches and lead-reviewed pull requests.

Native PHP uses explicit require_once statements and controller path matching. public/index.php opens the public PDO connection and dispatches only /api/public/ routes. admin/index.php opens the admin connection and dispatches only /api/admin/ routes. README.md (section 5) runs them on separate localhost ports; the earlier docs/how_to_run.md was removed on November 5.

Request Flow

HTTP client → public/index.php or admin/index.php → Controller → Service → Repository → PostgreSQL; images use storage/; notification services call TextBee or EmailJS.

Folder or file	Responsibility
public/index.php	Public API bootstrap (currently opens getAdminConnection()); also serves the public portal: app.html, css/ and js/.
admin/index.php	Admin API bootstrap and getAdminConnection(); also serves the marshal console: app.html, css/ and js/.
src/controllers/	Path/method matching, request extraction, JSON/image responses.
src/services/	Create/update workflows, reference codes, contact preferences, matching, search, images and notifications.
src/repositories/	Prepared database retrieval, inserts, updates and internal match cleanup.
src/db/db.php	Reads public/admin credentials from root .env and opens PDO connections.
src/integrations/	TextBee and EmailJS HTTP clients.
src/entities/ and src/utils/	Record entities, JSON/multipart parsing and shared validation/response helpers.

Folder or file	Responsibility
sql/	Schema, predefined-category data, demo data and sample role-creation script.
storage/	Uploaded items, lost-reports and claims image files; metadata stored in PostgreSQL.
docs/	Emptied on November 5 (only .gitkeep); setup guide now lives in README.md. Jenver maintains the separate documentation branch.

3. Database Design and Implementation

The current schema script declares eight tables: five core tables and three image tables. It adds contact_preference and contact checks to reports/claims, plus similarity_score and timestamps to potential_matches. Source inspection found a semicolon after each contact CHECK expression before the CREATE TABLE closing parenthesis. These two syntax defects must be corrected and a clean initialization rechecked before claiming reproducible installation. The schema script does not provision database logins or grants; since November 5 a separate sample script does. Both CHECK terminator defects are still present in revision 9a803d2.

Table	Primary key	Important fields and constraints
categories	category_id	name VARCHAR(100) NOT NULL; 11 predefined values in necessary-data script.
items	item_id	Category FK; title NOT NULL; description, found location/date/time; unclaimed/claimed/archived, default unclaimed.
lost_reports	report_id	Unique report_code; category FK; title/full_name required; nullable email/phone; contact_preference email/sms default email; at least one contact intended by CHECK; active/resolved/cancelled default active. Contact-CHECK statement terminator needs correction.
claims	claim_id	Unique claim_code; item FK; full_name required; nullable email/phone and contact_preference; at least one contact intended by CHECK; pending/approved/rejected default pending. Contact-CHECK statement terminator needs correction.
potential_matches	match_id	Report/item FKs, similarity_score NUMERIC(5,2) constrained 0–100, created_at. No unique pair constraint.
item_images	item_image_id	Required item FK; storage_key, mime_type and created_at.
lost_report_images	report_image_id	Required report FK; storage_key, mime_type and created_at.
claim_images	claim_image_id	Required claim FK; storage_key, mime_type and created_at.

Relationships and Deletion Behavior

Relationship	On deletion of referenced row
items.category_id → categories.category_id	SET NULL
lost_reports.category_id → categories.category_id	SET NULL
claims.item_id → items.item_id	CASCADE
potential_matches.report_id → lost_reports.report_id	CASCADE
potential_matches.item_id → items.item_id	CASCADE
item_images.item_id → items.item_id	CASCADE
lost_report_images.report_id → lost_reports.report_id	CASCADE
claim_images.claim_id → claims.claim_id	CASCADE

Categories relate to items and lost reports; items relate to claims; potential_matches links reports and items. Image rows have required foreign keys and cascade with their owning records. Core foreign-key columns remain nullable. The schema still does not enforce uniqueness of a potential-match report/item pair.

SQL Scripts and Initialization Order

- Create a new disposable PostgreSQL database named grp05_lostandfounddb.
- Resolve the two contact-CHECK statement terminators in the schema first, then run sql/grp05_lostandfounddb_script.sql with ON_ERROR_STOP enabled. This document records the issue; repository SQL was not changed during this update.
- For the predefined category list, run sql/grp05_lostandfounddb_necessary_data_script.sql after a successful schema initialization. Use the demo-data script (sql/grp05_lostandfounddb_demo_data_script.sql) only for deliberate demonstration on a disposable database. It replaced the old test-data script on November 5.
- As a PostgreSQL administrator, run sql/grp05_lostandfounddb_role_creation_sample_script.sql after replacing both enter_your_password_here placeholders with your own passwords. It creates lostandfound_app and lostandfound_public with their grants; use the same passwords in .env.

Important seed-data warning: the old test-data script (with TRUNCATE TABLE claims RESTART IDENTITY CASCADE) was removed on November 5. The replacement demo-data script deletes only its own records (claim codes starting DEMO-CLM-, report codes starting DEMO-LR-, item titles starting [DEMO]) before reinserting demo items, lost reports, claims and potential matches. It writes directly to the database, so it sends no SMS/email and inserts no image rows. The README still advises against running it repeatedly; use it only on a demo database.

4. Implemented API Reference

Public routes use http://127.0.0.1:8000; admin routes use http://127.0.0.1:8001. GET requests generally have no body. POST creation accepts JSON or multipart/form-data through getRequestData(); multipart images use images[]. PATCH and code-based view/cancel requests use JSON. In paired route entries below, the shorter second path uses the same /api/admin or /api/public prefix.

Method	Endpoint	Expected purpose
GET	/api/admin/test	Admin health message.
GET	/api/admin/items and /items/{id}	All items or one record, with image URLs.
POST	/api/admin/items/add	Create found item; JSON or multipart; optional images; matching follows.
PATCH	/api/admin/items/update/{id}	Partial item update and matching refresh/cleanup.
POST	/api/admin/items/{id}/images	Add images to an existing found item (multipart images[]; at most 5 per item). Added November 5.
DELETE	/api/admin/items/{id}/images/{image_id}	Delete one image from a found item and its stored file. Added November 5.
GET	/api/admin/lost-reports and /lost-reports/{id}	Lost-report collection or detail with image URLs.
PATCH	/api/admin/lost-reports/update/{id}	Partial report update and matching/contact handling.
GET	/api/admin/claims and /claims/{id}	Claim collection or detail with image URLs.
PATCH	/api/admin/claims/update/{id}	Partial claim update; relevant status notifications.
GET	/api/admin/categories and /categories/{id}	Category collection or detail.
GET	/api/admin/sms/stats and /sms/devices	TextBee gateway information wrappers.
GET	/api/admin/lost-reports/matches/{report_code}	Eligible matches for a report code.
GET	/api/admin/claims/images/{image_id}	Claim image binary response.
GET	/api/admin/items/images/{image_id} and /lost-reports/images/{image_id}	Item and lost-report image binary responses for the marshal console. Added November 5.
GET	/api/public/items	Unclaimed item collection with image URLs.
GET	/api/public/items/search	Filters: title, category_id, location_found, date_found, time_found.
GET	/api/public/categories and /categories/{id}	Category collection or detail.
POST	/api/public/lost-reports/add	Create report and images; find potential matches.
POST	/api/public/lost-reports/view	JSON report_code lookup.
POST	/api/public/lost-reports/cancel	JSON report_code; cancel an active report.
GET	/api/public/lost-reports/matches/{report_code}	Eligible match list with item images.
DELETE	/api/public/lost-reports/matches/{report_code}/{match_id}	Remove one potential match from a report identified by its code. Added November 5.
POST	/api/public/claims/add	Create claim for an unclaimed item; optional images/contact choice.
POST	/api/public/claims/view	JSON claim_code lookup; the response now includes the claimed item (claimed_item).
GET	/api/public/items/images/{image_id}	Found-item image binary response.
GET	/api/public/lost-reports/images/{image_id}	Lost-report image binary response.

Current Behavior and Limitations

- Collections return arrays. Public item browsing returns unclaimed items; search supports title, category_id, location_found, date_found and time_found. Filtering is not a general pagination feature. Search uses parameterized repository queries.
- POST and PATCH routes are now present. DELETE routes exist only for found-item images (admin) and potential matches (public); there is still no general record DELETE route, and obsolete potential-match rows are also deleted internally. Unsupported paths or methods generally return HTTP 404.
- Several detail routes still coerce suffixes to integers, for example 1abc to ID 1. Required-field, general-field, PATCH, date/time and contact-preference checks are implemented in RequestUtils; they should not be described as universally strict route-ID validation.
- Both controllers now strip query parameters before route matching. The public search route parses and validates its allowed query fields.
- Public/admin routes now use separate entry points and configurable database accounts, but no login or identity-based authorization middleware is present. Running a second port alone does not authenticate staff.

- SMSService sends TextBee notifications and EmailService calls EmailJS. Source paths cover claim creation/approval/rejection, report cancellation and potential-match notifications according to contact_preference. API and service wiring is implemented; successful live delivery has not been verified in this review.

Matching scores title (35%), description (25%), category (15%), location (15%) and date (10%), normalizing over available fields. Scores of at least 60 are stored. Eligible dates/statuses constrain candidates; relevant changes refresh or remove matches. Images accept JPEG, PNG or WebP, up to five per record and 5 MiB per image, with database metadata and separate stored files.

Frontend Interfaces (added November 5)

- Public portal (http://127.0.0.1:8000/app.html): campus lost-and-found landing page with a recently turned-in catalog, found-item detail dialog with enlarged photo preview, lost-report form, claim form that shows the item being claimed, and a reference-code tracker for reports and claims with cancellation, matches and match removal.
- Marshal console (http://127.0.0.1:8001/app.html): overview, found inventory (log, edit, image gallery upload/delete with enlarged preview), lost-report review with potential matches, and claim verification showing the claimed item.
- Both pages call the existing /api/public and /api/admin routes from public/js/app.js and admin/js/admin.js. No login screen or authentication exists, so the marshal console is open to anyone who can reach port 8001.

5. Application Roles and Intended Database Access

The supplied member roles and responsibilities remain unchanged. Intended student/guest, staff/marshal and DBA responsibilities are distinguished from actual application authorization and PostgreSQL privileges.

Application role	Intended responsibility	Intended database access
Student / guest	Browse found items and categories; submit reports and claims; cancel own submission by reference code.	SELECT on permitted public data; scoped INSERT for submissions; cancellation requires a constrained design.
Campus staff / marshal	Register found items, manage categories, inspect reports/claims, and update statuses.	SELECT, INSERT, and UPDATE only on required objects; sequence USAGE for generated IDs.
DBA / database owner	Initialize schema, provision accounts, maintain database objects, and handle backup/restore.	Separate administrative account; not the ordinary application login.
Application runtime account	Connects the PHP application to PostgreSQL.	Only the database operations required by implemented application code.

Actual Enforcement Status

Connection::getPublicConnection() reads DB_PUBLIC_USER and DB_PUBLIC_PASSWORD; getAdminConnection() reads DB_USER and DB_PASSWORD. admin/index.php uses getAdminConnection(). public/index.php was changed on November 5 (commit ead406a) to call getAdminConnection() as well, so the public API currently runs with the DB_USER account (lostandfound_app) instead of the narrower DB_PUBLIC_USER account. No user-account table or login middleware is present. The schema script does not create roles; the new sample script sql/grp05_lostandfounddb_role_creation_sample_script.sql creates lostandfound_app (SELECT/INSERT/UPDATE on all tables, DELETE on potential_matches and the three image tables) and lostandfound_public (SELECT on six tables, INSERT on reports, claims, matches and report/claim images, UPDATE of lost_reports.status, DELETE on potential_matches). These grants have not been applied or tested in this update.

Earlier local rehearsals used a non-superuser runtime login. Those results predate the latest public/admin split and are not verification of the new account permissions. Keep real database passwords, TextBee keys and EmailJS credentials out of shared documents and Git.

6. Development Progress and Milestones

The dated milestones below are grounded in main history through November 4, 2026. Feature presence is based on source inspection; historical local checks remain tied to their original dates and revisions.

Date	Milestone or revision	Recorded progress
Oct 30, 2026	5aaac06 / PR #23	Administrative retrieval merged; entity foundations added in 17dfae5.
Oct 31, 2026	549aefd / PR #25	Sanitized JSON database-error handling merged.
Nov 1, 2026	2778937; 5476cf8 / PR #26	Found-item POST and public report/claim submission added.
Nov 2, 2026	6ab5a11 / PR #28; 9800f0a / PR #29	Readable year-based codes and earlier schema typo fix merged.
Nov 3, 2026	f889c9c; 515d7b0; 8f60d53	Item/report/claim PATCH workflows and status checks.
Nov 4, 2026	008fa79; 21fb666	Potential matching operations and refresh logic.
Nov 4, 2026	7d56a8b; 46f0d99; c7c7976	Contact preference plus SMS and EmailJS notification wiring.
Nov 4, 2026	c5b05aa; b15c8e3	Unclaimed-item search and image workflows.
Nov 4, 2026	dd4b03d; 78ab3d9	Separate public/admin connections and entry points; predefined category seed.
Nov 4, 2026 (late)	6f461f2; 5e78830	README environment variables and notification requirements clarified (11:32-11:35 PM).
Nov 5, 2026	9d215e0; 6416d36	Database role-creation script and README role setup; TextBee/EmailJS listed as prerequisites (1:03-1:59 AM).
Nov 5, 2026	f74c60f; ead406a; 0f271d2; 31b4bd5; 77d3d5e; ba3bf5b; PR #44	Redesigned public and marshal frontends built for issue #41 and aligned with the current matching, report and image APIs; backend synced with main (2:26-2:52 AM).
Nov 5, 2026	c4a629b; 0962465; 35098d0	Frontend formatting; claim-verification and lost-report review forms fixed; dynamic admin form handling corrected (4:34-4:41 AM).
Nov 5, 2026	d06e2dd; 4a4d902; 0a09202; 5ed86b3; 359a6be; ee85270; 2b4fd18	Admin item-image upload/delete endpoints and routes; admin image retrieval for items and lost reports; public found-item detail dialog; editable found-item details with enlarged photo preview (4:48-5:16 AM).
Nov 5, 2026	f87edf1; 68ce720; fedf499; e43b26b	Claimed-item details shown in the claim form, public tracking and admin claim review; campus location dropdown for lost-report review (5:18-5:31 AM).
Nov 5, 2026	6ae875f; 234ad4b; fdfba67; 4044290; PR #46	Demo seed data script added; outdated test-data script removed; docs folder cleared; notification wording updated; PR #46 merged 5:39 AM.
Nov 5, 2026	e1920b1; PR #47	README note on notification provider setup assistance; PR #47 merged 5:55 AM.
Nov 5, 2026	658819e; a19dac9; f15d026; 0832d4a; 3718963	Public potential-match removal: DELETE route, service, repository method and interface (6:01 AM).
Nov 5, 2026	ff6a97e; 9a803d2	README simplified for public setup and updated (11:37-11:43 AM); latest reviewed revision 9a803d2.

Contribution Record

The cover table retains the supplied names and roles. The contribution table remains editable for member-confirmed work and commit/PR evidence; this update does not infer individual ownership from role titles.

Member	Actual work completed	Total Commits
Daniel Cueto	Back-End Developing	106
Emmanuel Manansala	Front-End Developing/Documentation	28
Aaron Paul Imperial	Front-End Developing	19
Justine Philip Raguin	Back-End Developing/Documentation	16
Jenver Napenas	Documentation	14

7. Testing and Verification Evidence

Historical results below describe the revisions actually tested. For this November 4 update, the current main source was inspected and all 26 PHP files passed syntax checks. The complete latest API workflow was not rerun, live notifications were not sent, and cross-group deployment was not certified. The November 5 update repeated the syntax checks on revision 9a803d2 (26 PHP and 3 JavaScript files passed) and did not rerun the complete workflow either.

Verification activity	Recorded result	Scope and limit
Earlier retrieval regressions	36 requests / 69 assertions passed on earlier October/November revisions.	Historical behavior; not latest-main test coverage.
Nov 1 submission checks	119 HTTP scenarios; 19 PHP syntax checks on PR #26 review revision.	Before later matching, image and public/admin changes.
Nov 2 code checks	2,000 generated codes; 43 HTTP scenarios; 69 regression assertions.	Readable-code revision; existing codes preserved.

Verification activity	Recorded result	Scope and limit
Nov 3 local dependency check	mbstring enabled locally; Unicode length and invalid-email validation passed.	Older checkout; not a full November 4 integration test.
Nov 4 PHP syntax	26 of 26 source PHP files passed lint.	Source syntax only; does not exercise SQL or delivery.
Nov 4 schema review	Two misplaced CHECK terminators identified in source. Database execution unavailable because local project PostgreSQL was not running.	No fresh-install success claimed; no existing data modified.
Nov 5 syntax and source review	26 of 26 PHP and 3 of 3 JavaScript files passed syntax checks on 9a803d2; both schema CHECK terminators still present.	Syntax and source inspection only; SQL not executed, new pages not exercised in a browser, no live delivery.
Latest matching/images/notifications	Source wiring inspected; complete latest workflows and live delivery not rerun.	Requires isolated integration and delivery verification.
Cross-group verification	No completed results supplied.	Actual other-project tests must be recorded separately.

Verification Limits

- Earlier checks and the November 4 syntax pass are local evidence. They do not prove another group can independently initialize and run the latest revision.
- Creation, PATCH, matching, search, images and notification calls now exist in source. Their complete latest-version behavior, photo persistence, delivery and role permissions still require integration testing; the new public and marshal interfaces have not been exercised in a browser in this update.
- No completed cross-group results are supplied with this document. Leave result rows unfilled until an actual activity has been performed and evidence recorded.
- PR #25 is merged (October 31); PR #26 is merged (November 1); PRs #28 and #29 are merged (November 2). Earlier test counts are historical and must not be presented as test coverage of the November 4 revision.

Evidence to Attach Before Submission

- Capture genuine public/admin API screenshots showing GET, POST, PATCH, code lookup/cancellation, search, matches and images after the latest setup works.
- Capture screenshots of the public portal (catalog, item detail, lost-report form, claim form, tracking page) and the marshal console (overview, found inventory with image gallery, lost reports, claim verification) at http://127.0.0.1:8000/app.html and http://127.0.0.1:8001/app.html, with credentials hidden.
- A screenshot or export showing the database tables and sample records in PostgreSQL/pgAdmin.
- Screenshots of the project running locally, with credentials and private keys hidden.
- Record genuine cross-group activity outcomes in the existing Activity Log tab: one row per performed Activity 3, 4, 5 or 6 on a project, with evidence and Success / Some issues / Failed as appropriate.

The supplied document contains no application screenshots. None are invented in this update. Add evidence captured from a working installation with credentials hidden.

8. Installation and Configuration

These steps reflect the split public/admin API and interfaces in main revision 9a803d2. Resolve the documented schema syntax defects and verify account grants before treating this as a successful clean-install procedure.

Prerequisites

- Git and access to the repository.
- PHP 8.0 or later (per README) with pdo_pgsql, pgsql, curl, mbstring and fileinfo enabled. Image validation also uses getimagesize(). Configure upload_max_filesize/post_max_size consistently with the application image limits.
- PostgreSQL server and a client such as pgAdmin or psql.
- Postman, Thunder Client, or another HTTP client.

Clone the Repository

```
git clone https://github.com/grp05-dba/grp05_lostandfound.git
cd grp05_lostandfound
```

Create and Initialize the Database

Create a fresh database. Correct the two invalid CHECK terminators, initialize tables, and then load predefined categories. Do not run these commands against data you need to preserve:

```
psql -h localhost -p 5432 -U your_database_owner -d postgres -c "CREATE DATABASE grp05_lostandfounddb;"
psql -h localhost -p 5432 -U your_database_owner -d grp05_lostandfounddb -v ON_ERROR_STOP=1 -f
sql/grp05_lostandfounddb_script.sql
psql -h localhost -p 5432 -U your_database_owner -d grp05_lostandfounddb -v ON_ERROR_STOP=1 -f
sql/grp05_lostandfounddb_necessary_data_script.sql
```

Create the Local .env File

Use .env.example as the list of expected settings. Create separate public/admin PostgreSQL logins and grant only the permissions each workflow needs. The schema script does not do this automatically; use the role-creation sample script from Section 3. The placeholders below are not working credentials:

```
DB_HOST="localhost"
DB_PORT=5432
DB_NAME="grp05_lostandfounddb"
DB_USER="your_admin_application_login"
DB_PASSWORD="your_admin_local_password"
DB_PUBLIC_USER="your_public_application_login"
DB_PUBLIC_PASSWORD="your_public_local_password"
TEXTBEE_API_KEY="your_local_key"
TEXTBEE_BASE_URL="https://api.textbee.dev"
EMAILJS_BASE_URL="https://api.emailjs.com"
EMAILJS_SERVICE_ID="your_service_id"
EMAILJS_TEMPLATE_ID="your_template_id"
EMAILJS_PUBLIC_KEY="your_public_key"
```

Start and Test the API

```
Public API terminal:
php -S 127.0.0.1:8000 -t public

Admin API terminal:
php -S 127.0.0.1:8001 -t admin
```

With the server running, test the following URLs in Postman or Thunder Client:

```
GET http://127.0.0.1:8000/api/public/categories
GET http://127.0.0.1:8000/api/public/items
GET http://127.0.0.1:8001/api/admin/test
```

JSON responses are expected for the API URLs above. The same servers also serve the interfaces at http://127.0.0.1:8000/app.html (public portal) and http://127.0.0.1:8001/app.html (marshal console). TextBee and EmailJS credentials are optional; without them the core application works but notifications are not delivered. Both terminals must stay open. Do not send the admin health path to the public process. Stop each PHP server with Ctrl+C.

9. Issues, Limitations, and Remaining Work

Issue or gap	Current status	Next action
Frontend and login	No complete frontend/login middleware in reviewed main.	Add staff authentication, browser-test every screen and capture screenshots.
Create/update coverage	POST and PATCH exist; DELETE only for item images and potential matches; no general record DELETE route.	Test latest workflows and document their inputs/responses.
Database initialization	Contact CHECK clauses contain invalid semicolon terminators (still present in 9a803d2).	Correct/review SQL and verify clean initialization before seeding.
Public/admin privileges	Sample role script added November 5; public/index.php currently uses the admin connection.	Decide whether the public process should use DB_PUBLIC_USER, run the role script and test allowed/denied actions.
Matching and notifications	Integrated weighted matching and preference-based service calls.	Test trigger paths, refresh/cleanup and actual delivery with approved credentials.
Photo storage	Eight tables declared; images stored as files outside web roots.	Test uploads/rollback/retrieval; include image storage in backup plans.

Issue or gap	Current status	Next action
Predefined values	11 category values seeded; location fields remain free-text in the database, while the found-item and lost-report review forms now offer a campus location dropdown.	Confirm campus location labels and provide consistent UI choices; no locations table is declared.
Cross-group installation	No completed external results supplied.	Perform required activity attempts; record outcomes and evidence in Activity Log.

Troubleshooting Notes

- If PHP reports that it cannot find a driver, verify pdo_pgsql is enabled in the php.ini used by the CLI/server.
- For database connection errors, check PostgreSQL, the database name, host/port and the correct public/admin credentials and grants in .env.
- If initialization fails near a semicolon in a contact CHECK constraint, resolve the schema terminators before rerunning on a fresh disposable database. Do not continue loading seed data after failed DDL.
- If sample data produces duplicate-key errors, do not repeatedly rerun the script; initialize a fresh disposable database if appropriate.
- If an endpoint returns 404, check the exact method and route path in the API reference.

10. Cross-Group Verification

Project / group tested	Test Result/Branches/Repository/Feedbacks/URL
Group 2	https://github.com/grp05-dba/grp05_lostandfound/blob/main/group-test-results/activity-3/grp02-parkit/testing-output.md
Group 3	https://github.com/grp05-dba/grp05_lostandfound/blob/main/group-test-results/activity-3/grp03-Project-OJT-RMA-System/testing-output.md
Group 4	https://github.com/grp05-dba/grp05_lostandfound/blob/main/group-test-results/activity-3/grp04-brewpos/testing-output.md

11. AI Usage Declaration

OpenAI Codex and Claude.ai assisted with implementation support, source/history inspection, local checks and documentation updates. The supplied AI declaration below is retained. The group remains responsible for source review, testing, understanding the work and approving submission.

AI tool	Tasks recorded in repository	Validation / responsibility
OpenAI Codex	Source and Git history inspection; documentation drafting; local verification support.	The group remains responsible for reviewing generated content, checking it against the source code and SQL, testing the application, and understanding submitted work.
Claude.ai	Source, Guide, Documentation, Drafting	The group remains responsible for ideas, reviewing of generated contents, sought only guides and understanding of the project

12. Conclusion and Submission Checklist

PUP-SPOT now has a broader PHP/PostgreSQL API: record creation and PATCH updates, reference-code workflows, matching, search, image handling and notification integration, with eight declared tables and separate public/admin entry points. Main revision 9a803d2 is the source baseline for this update, and it adds browser interfaces, admin image management, potential-match removal and a role-creation script. A clean schema initialization, latest end-to-end and browser tests, authentication, live notification delivery and cross-group results remain to be verified before claiming the full activity is complete.

Repository and Documentation Links

- Project repository: https://github.com/grp05-dba/grp05_lostandfound
- Documentation branch: https://github.com/grp05-dba/grp05_lostandfound/tree/docs/project-documentation
- Current main setup README: https://github.com/grp05-dba/grp05_lostandfound/blob/9a803d2771610166e638eeba24321764377de915/README.md
- Role-creation sample script: https://github.com/grp05-dba/grp05_lostandfound/blob/9a803d2771610166e638eeba24321764377de915/sql/grp05_lostandfounddb_role_creation_sample_script.sql
- Frontend sources: https://github.com/grp05-dba/grp05_lostandfound/tree/9a803d2771610166e638eeba24321764377de915/public and https://github.com/grp05-dba/grp05_lostandfound/tree/9a803d2771610166e638eeba24321764377de915/admin
- Current schema: https://github.com/grp05-dba/grp05_lostandfound/blob/9a803d2771610166e638eeba24321764377de915/sql/grp05_lostandfounddb_script.sql
- Reviewed implementation history: https://github.com/grp05-dba/grp05_lostandfound/commits/9a803d2771610166e638eeba24321764377de915

13. Development History and Content Changes

This section presents the development history of the PUP-SPOT Lost and Found System based on the commit history of the group's GitHub repository. It records the major improvements, features, corrections, and documentation updates made from the initial repository setup up to November 5, 2026.

13.1 October 26, 2026 – Initial Repository Setup

The project repository was established on October 26, 2026. The initial development focused on preparing the project structure, configuring database connectivity, and organizing the files needed for the system.

The first commit was titled "Initial Commit" (6bae603).

The initial updates included:

- Creating the initial project structure.
- Preparing the database connection using PDO.
- Adding and organizing the database role creation script.
- Updating the .gitignore file.
- Preparing folders for entities, integrations, repositories, and services.
- Establishing the foundation for the Public and Admin components.

13.2 October 27, 2026 – Core System Structure and Item Management

Development continued with the implementation of the system's basic architecture and item management functionality.

The group worked on the Public and Admin controllers, repositories, and services to establish how the system processes requests and manages item records.

Major updates included:

- Implementing Public and Admin controllers.
- Establishing request handling using HTTP methods and URL paths.
- Improving dependency loading and repository organization.
- Implementing the initial item management functionality.
- Adding and improving database schema files.
- Updating the README and project resources.

These changes established the foundation for the system's backend operations.

13.3 October 28, 2026 – Category Management and Notification Integration

The system's backend functionality was expanded to support category management and external notification services.

Major updates included:

- Implementing the Category Repository and Category Service.
- Adding category retrieval functionality.
- Refactoring services and controllers to use the Item Service.
- Integrating the TextBee API for SMS functionality.
- Adding administrative endpoints for SMS statistics and device information.
- Updating the .gitignore file to exclude temporary testing files.
- Improving database and controller organization.

These updates helped establish the system's category management and notification capabilities.

13.4 October 29, 2026 – Administrative Retrieval and Email Integration

The group continued improving the administrative side of the system and introduced additional notification functionality.

Major updates included:

- Implementing administrative retrieval of lost reports, items, and claims.
- Adding the EmailJS client for sending emails through the EmailJS API.
- Improving the documentation folder and project organization.
- Documenting database role mapping and project progress.
- Keeping coursework documents separate from the actual application files.
- Applying corrections based on code review feedback.

These improvements expanded the system's administrative functionality and prepared it for email notification integration.

13.5 October 30, 2026 – Entity Refactoring and Documentation Improvements

The development process focused on improving the organization of the backend and making the system's data handling more structured.

Major updates included:

- Introducing entity classes for Claim, Item, and LostReport.
- Refactoring item retrieval logic.
- Improving the handling and conversion of database record identifiers.
- Adding documentation describing the API, architecture, database, and branch workflow.
- Updating the repository documentation address to the group's organization repository.
- Improving database error handling.
- Returning JSON-formatted server error responses when database exceptions occur.

These changes improved the maintainability of the backend and made its structure easier to understand.

13.6 October 31, 2026 – Public Submission Functionality

The system began expanding its public-facing functionality by introducing submission endpoints for lost reports and claims.

Major updates included:

- Implementing public lost report submission.
- Implementing public claim submission.
- Improving submission validation and reference code generation.
- Applying review feedback to the submission process.
- Merging the public submission functionality through Pull Request #26.

These updates allowed the public-facing system to begin supporting the submission of lost item reports and claims.

13.7 November 1, 2026 – Validation and Reference Code Improvements

Development focused on strengthening the validation process and improving the reliability of submitted records.

Major updates included:

- Improving request validation methods.
- Enhancing error handling in services and utility classes.
- Shortening claim and report reference codes using a year and six-digit format.
- Correcting a database schema issue that prevented proper schema creation.
- Improving item update functionality.
- Adding PATCH functionality for updating existing records.
- Improving claim code uniqueness validation.

These changes helped make the system's data processing more consistent and reliable.

13.8 November 3, 2026 – Update Operations, Potential Matching, and Image Support

A major development period focused on expanding the system's functionality through record updates, potential item matching, and image handling.

Record update functionality:

- Implementing PATCH endpoints for lost reports and claims.
- Adding repository methods for partial record updates.
- Improving validation of updated information.
- Ensuring claim codes remain unique.
- Correcting nullable field handling and allowed status values.
- Applying code review feedback to the update operations.

Potential matching functionality:

- Implementing potential match operations.
- Connecting automatic potential matching to the system.
- Adding a browser-based Fuse.js helper for client-side matching.
- Improving matching-related API documentation.
- Merging the Fuse.js potential matching feature through Pull Request #37.

Image handling functionality:

- Implementing database-backed image storage.
- Adding photo upload and retrieval endpoints.
- Connecting uploaded images to their corresponding records.
- Restricting image retrieval to the appropriate owner records.
- Documenting photo retrieval routes and PHP upload limitations.
- Merging the image-related implementation through Pull Request #38.

Notification integration:

- Integrating the TextBee and EmailJS notification functionality.
- Improving the potential matching notification process.
- Supporting SMS and email notifications based on user preferences.
- Merging the notification implementation through Pull Request #36.

These updates significantly expanded the system's backend capabilities and improved how records, images, potential matches, and notifications are handled.

13.9 November 4, 2026 – Frontend Redesign and Feature Improvements

November 4 involved extensive improvements to the frontend, backend integration, item management, and notification functionality.

Public and Admin interface improvements:

- Redesigning the Public and Marshal interfaces through the frontend redesign work.
- Aligning frontend components with the updated backend APIs.
- Improving the display of potential matching information.
- Correcting frontend form handling and event listeners.
- Improving claim verification and lost report review forms.
- Updating the administrative review interface.
- Merging the frontend redesign through Pull Request #46.

Item and image management:

- Adding image upload and deletion endpoints for administrative item management.
- Implementing an item image gallery and management controls.
- Adding detailed views for public catalog items.
- Adding a public found-item detail dialog.
- Improving the image preview and item detail interfaces.
- Allowing found item details to be edited.
- Improving image URL handling and retrieval for both interfaces.
- Adding an administrative lost report image retrieval endpoint.

Claim and tracking improvements:

- Adding claimed item details to the claim submission form.
- Displaying claimed item information in public tracking and administrative claim review.
- Improving the layout of claimed item details.
- Adding a campus location dropdown for lost report review.
- Improving the administrative potential matches panel.
- Adding the ability to remove public potential matches.

Database and documentation improvements:

- Adding a comprehensive demonstration seed data script.
- Removing outdated test data scripts.
- Improving notification messages.
- Updating database role management instructions.
- Clarifying TextBee and EmailJS prerequisites.
- Updating environment variable configuration and API setup instructions.
- Adding documentation for notification provider setup through Pull Request #47.

These changes improved the overall system experience and brought the frontend closer to the functionality provided by the backend.

13.10 November 5, 2026 – Database Separation, Architecture, and Documentation Updates

November 5 focused on refining the database connection structure, improving documentation, and correcting the Public and Admin application configurations.

Database and application structure:

- Introducing a separate public database connection.

- Updating the Admin API structure.
- Adding a separate Admin entry point.
- Updating the database schema and role configuration.
- Adding initial category entries to the database.
- Correcting extra semicolons in scripts.
- Improving the README and setup instructions.

Public and Admin access model:

- Clarifying the separate responsibilities of the Public and Admin applications.
- Using the restricted lostandfound_public database role for the Public application.
- Using the lostandfound_app database role for administrative application operations.
- Documenting the intended separation between the publicly accessible application and the campus-restricted administrative API.
- Retaining the existing tested database permissions and role queries.

Documentation updates:

- Improving the project documentation index.
- Updating API documentation.
- Revising the system architecture overview.
- Updating the database reference.
- Improving setup instructions.
- Clarifying Public and Admin user roles.
- Updating the development history and repository review.
- Improving branch workflow documentation.

The documentation updates were prepared in the docs/project-documentation branch and later integrated into main through Pull Request #51.

Final connection correction:

The commit f36295f, titled "Fixed connection in public index," corrected the database connection used by the public application. The Public entry point was updated to use Connection::getPublicConnection() instead of Connection::getAdminConnection().

The latest recorded repository update on November 5 was 5d38a55, titled "added folders for testing."

13.11 Summary of Development Progress

From October 26 to November 5, 2026, the PUP-SPOT Lost and Found System progressed from its initial repository and database setup into a more complete application with public submissions, administrative record management, image handling, potential matching, notification integration, and improved documentation.

The major development milestones included:

1. Initial project structure and database configuration.

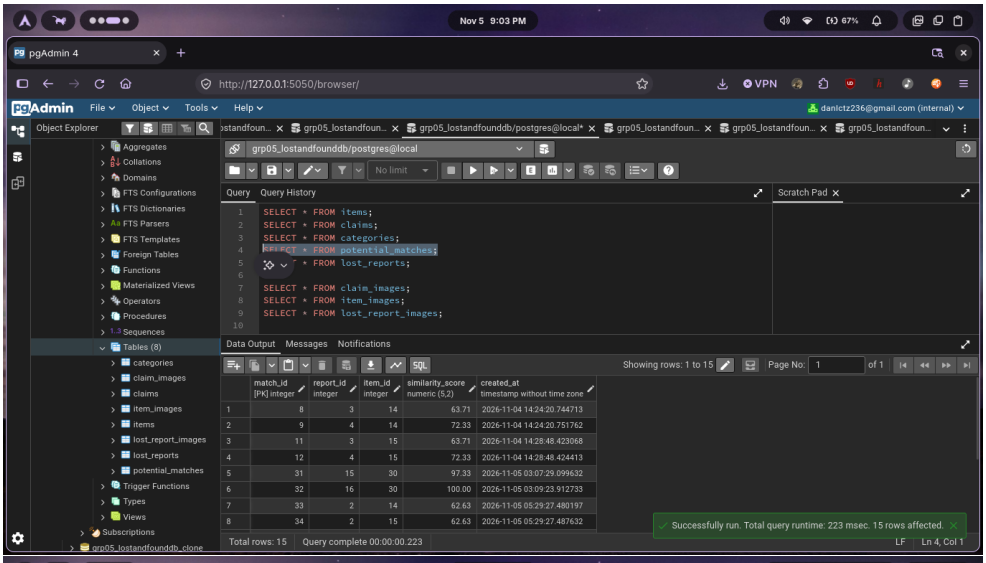
2. Core backend architecture and item management.
3. Category management and notification integration.
4. Administrative retrieval and public submission functionality.
5. Validation, reference code generation, and record update operations.
6. Potential matching and image upload and retrieval.
7. Public and Admin frontend improvements.
8. Database connection separation and role configuration.
9. Documentation improvements and repository integration.

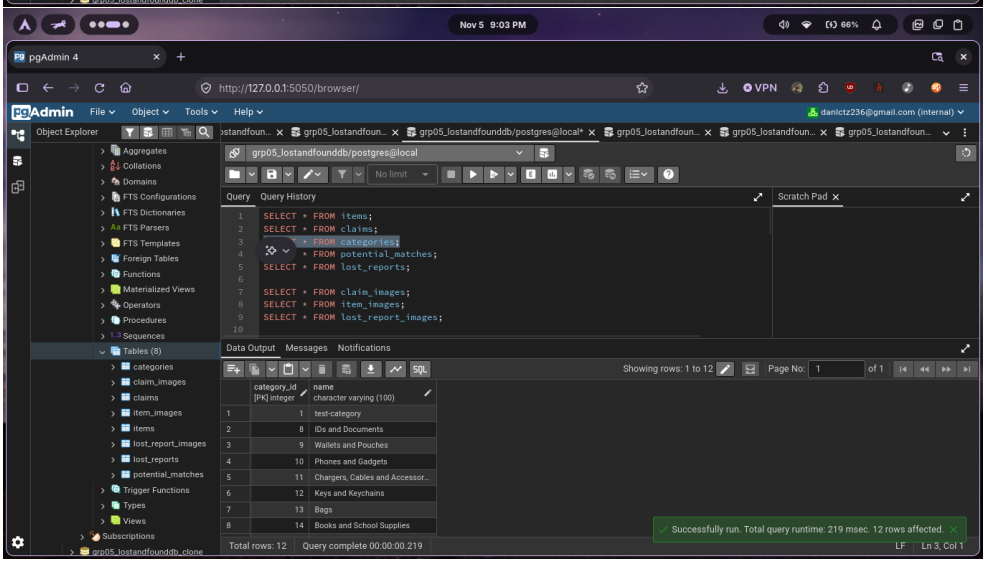
As of November 5, 2026, the repository's latest reviewed source revision is f36295f, followed by the testing-folder update 5d38a55. The documentation branch has also been merged into main through Pull Request #51.

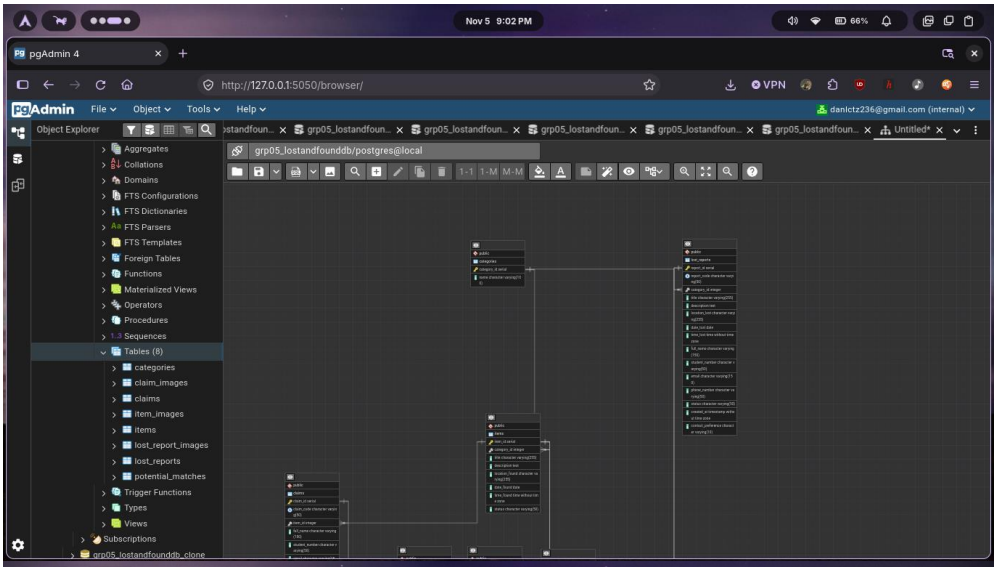
Repository: https://github.com/grp05-dba/grp05_lostandfound

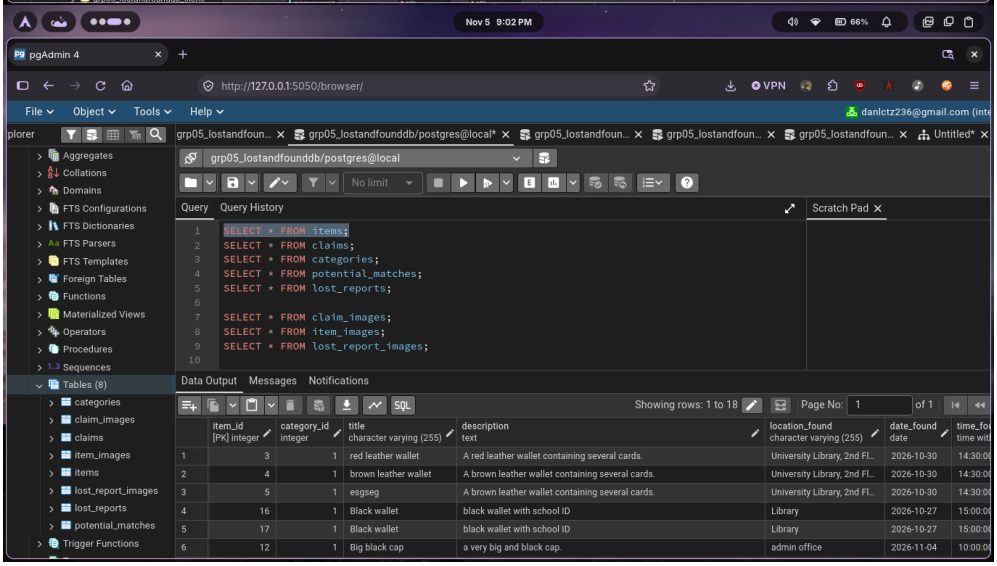
Important commits and integration references:

- Initial Commit: 6bae603
- Initial backend implementation: a1e65e5
- Public submission implementation: a6c305c
- Potential matching and image support: November 3 commits
- Frontend redesign: Pull Request #46
- Notification setup documentation: Pull Request #47
- Public and Admin database connection updates: dd4b03d
- Documentation integration: Pull Request #51
- Latest Public connection correction: f36295f
- Latest repository update: 5d38a55


pgAdmin 4
http://127.0.0.1:5050/browser/
grp05_lostandfounddb/postgres@local
SELECT * FROM items;
SELECT * FROM claims;
SELECT * FROM categories;
SELECT * FROM potential_matches;
SELECT * FROM lost_reports;
SELECT * FROM claim_images;
SELECT * FROM item_images;
SELECT * FROM lost_report_images;
Data Output
Messages
Notifications
Total rows: 15
Query complete 00:00:00.223
Successfully run. Total query runtime: 223 msec. 15 rows affected.


pgAdmin 4
http://127.0.0.1:5050/browser/
grp05_lostandfounddb/postgres@local
SELECT * FROM items;
SELECT * FROM claims;
SELECT * FROM categories;
SELECT * FROM potential_matches;
SELECT * FROM lost_reports;
SELECT * FROM claim_images;
SELECT * FROM item_images;
SELECT * FROM lost_report_images;
Data Output
Messages
Notifications
Total rows: 12
Query complete 00:00:00.219
Successfully run. Total query runtime: 219 msec. 12 rows affected.

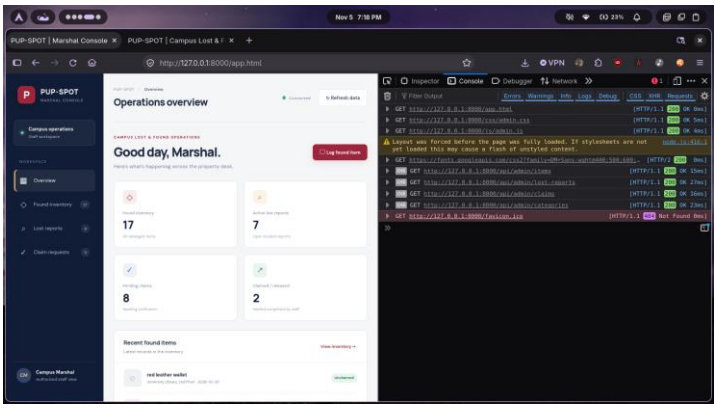




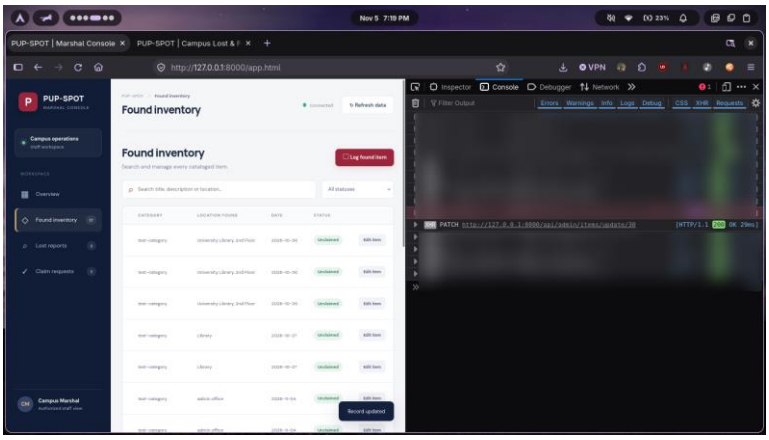
Screenshots
Admin home page with endpoints:
GET /api/admin/items

GET /api/admin/lost-reports

GET /api/admin/claims
GET /api/categories



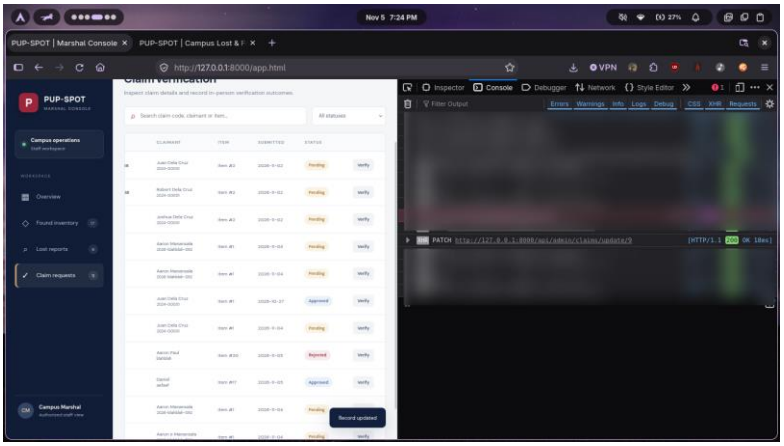
Admin items page with endpoint:
PATCH /api/admin/items/update/{item_id}



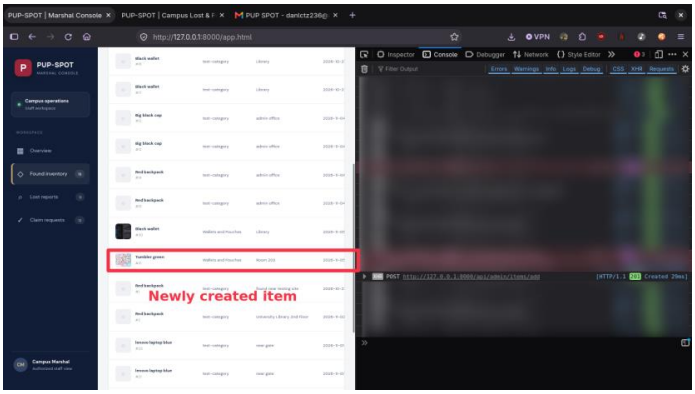
Admin lost reports page with endpoinst:
GET /api/admin/lost-report/matches/{report_code}

PATCH /api/admin/lost-reports/update/{item_id}

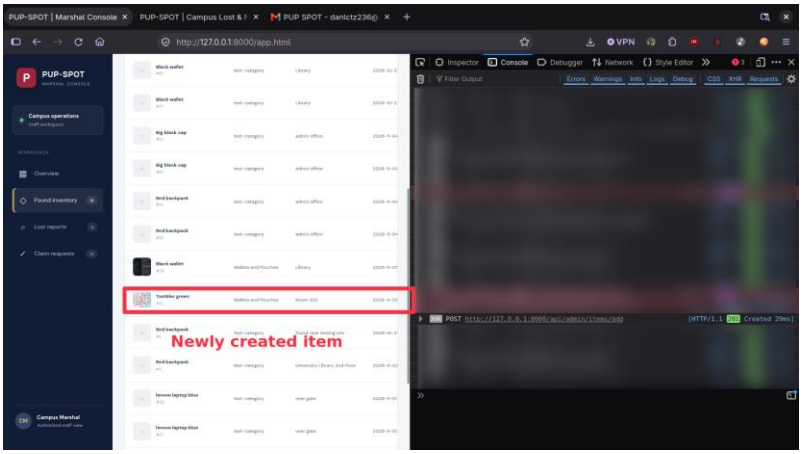
Admin claims page with endpoint:
PATCH /api/admin/claims/update/{item_id}



Admin found items page with endpoint:
POST /api/admin/items/add

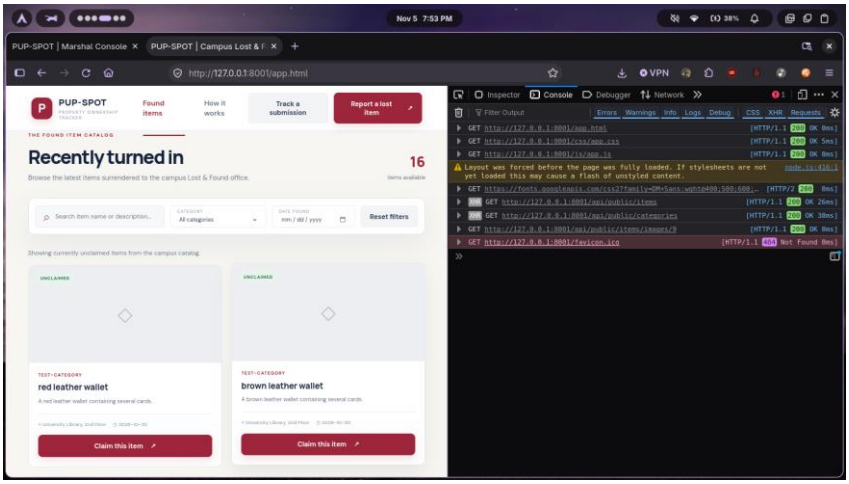


Admin create items page with endpoint:
POST /api/admin/items/add



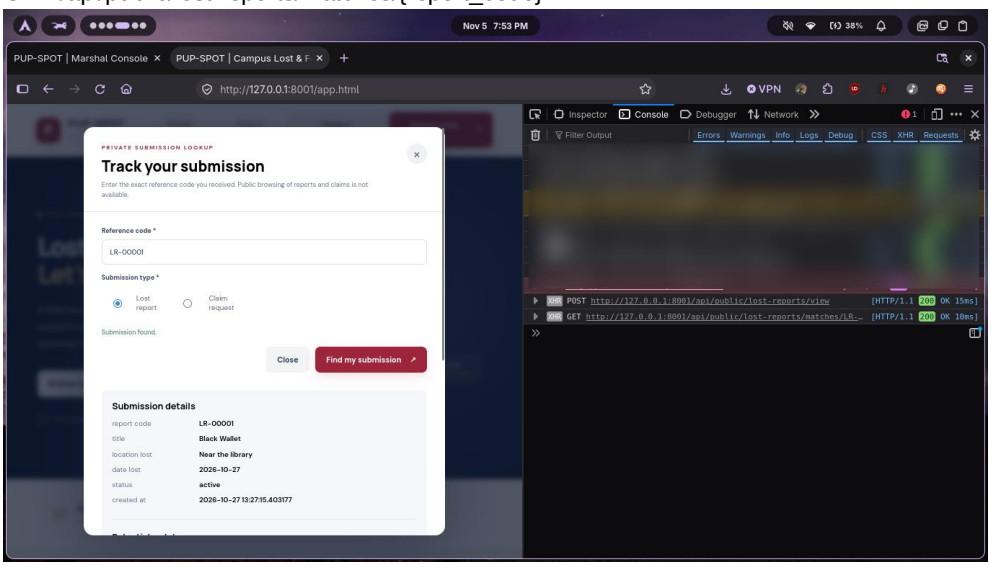
Public homepage with endpoints:
GET /api/public/items

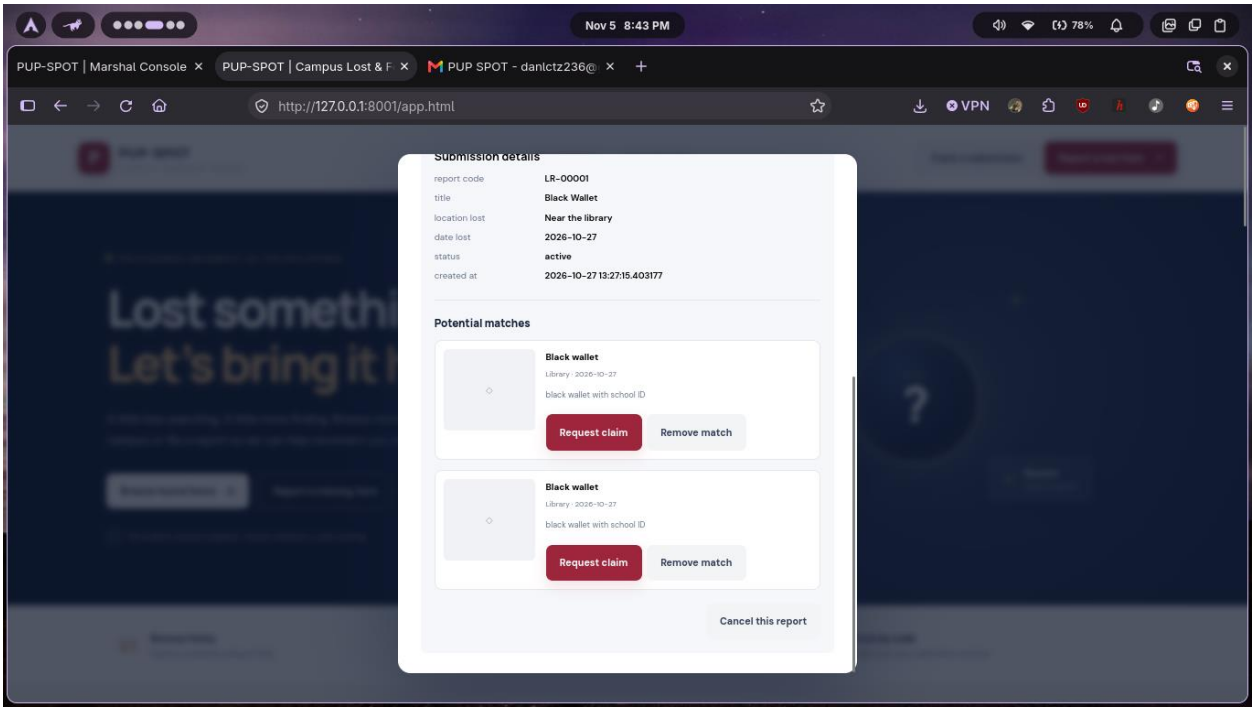
GET /api/public/categories
GET /api/public/item/images/{item_id}



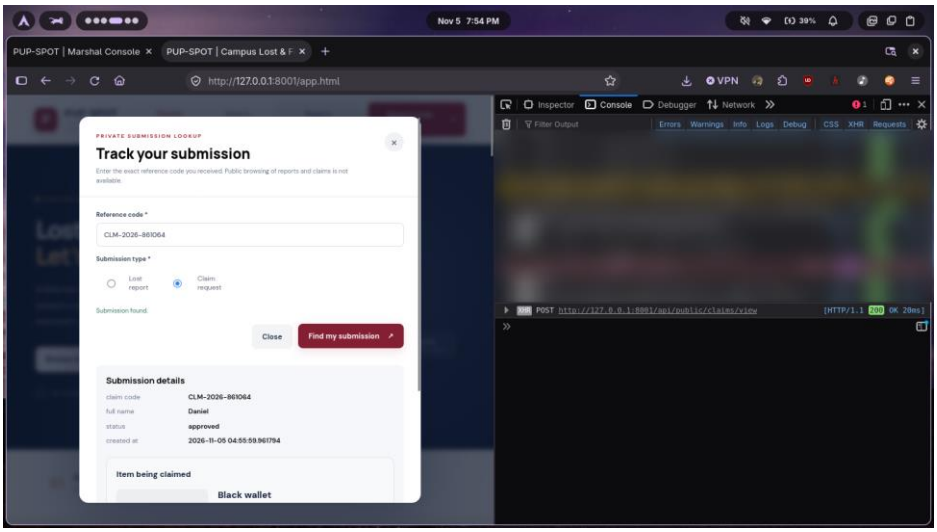
Public track lost report submission with endpoint:

GET /api/public/lost-reports/view
GET /api/public/lost-reports/matches/{report_code}



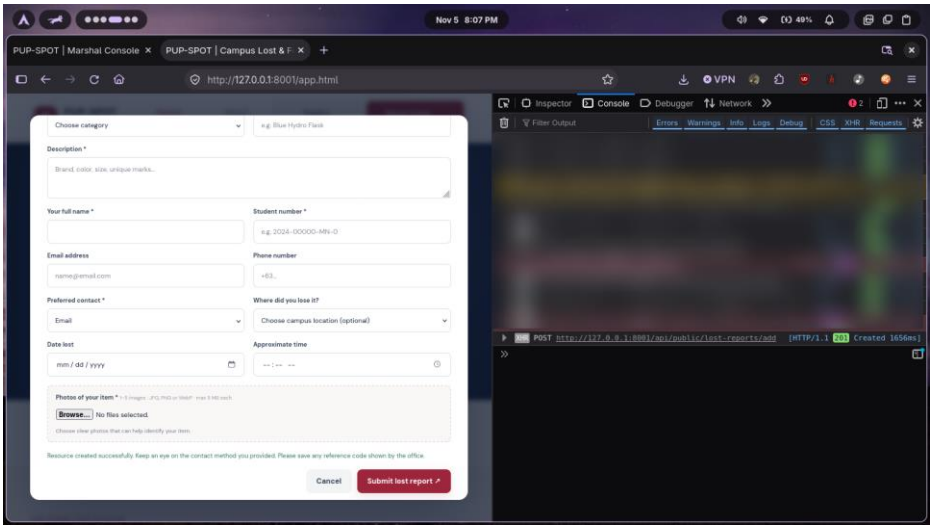


Public track claim submission with endpoint:
POST /api/public/claims/view



Public add lost report with endpoint:

POST /api/public/lost-reports/add



Notification when creating lost reports:

From:

